

Section 9.2 - Image segmentation

December 2, 2025

1 Image segmentation

- 1) Semantic segmentation: assign a class to every pixel - independently classified
- 2) Other task: instance segmentation - parse out individual object instances

1.1 Oxford-IIIT Pets dataset

- 7,390 images (JPG) & foreground-background masks (PNG)
- Masks = (trimap) labels: 1-foreground, 2-background, & 3-contour

```
[1]: # Prepare input file paths
import os

input_dir = "images/"
target_dir = "annotations/trimaps/"

input_img_paths = sorted(
    [os.path.join(input_dir, fname)
     for fname in os.listdir(input_dir)
     if fname.endswith(".jpg")])
target_paths = sorted(
    [os.path.join(target_dir, fname)
     for fname in os.listdir(target_dir)
     if fname.endswith(".png") and not fname.startswith(".")]])
```

```
[2]: # inspect & visualize
import numpy as np

print(np.array(input_img_paths).shape)
print(np.array(target_paths).shape)
```

```
(7390,)
(7390,)
```

```
[3]: import matplotlib.pyplot as plt
from tensorflow.keras.utils import load_img, img_to_array

plt.axis("off")
```

```
plt.imshow(load_img(input_img_paths[10]))
```

```
[3]: <matplotlib.image.AxesImage at 0x7fab68a3340>
```



- Normalize label values: 0, 1, 2

```
[4]: # Load inputs & targets into Numpy arrays
def display_target(target_array):
    normalized_array = (target_array.astype("uint8") - 1) * 127
    plt.axis("off")
    plt.imshow(normalized_array[:, :, 0])

img = img_to_array(load_img(target_paths[10], color_mode="grayscale"))
display_target(img)
```



- Same random seed: ensure the same order of inputs & targets
- Training & 1K samples for validation

```
[5]: import numpy as np
import random

img_size = (200, 200)
num_imgs = len(input_img_paths)

random.Random(1337).shuffle(input_img_paths)
random.Random(1337).shuffle(target_paths)

def path_to_input_image(path):
    return img_to_array(load_img(path, target_size=img_size))

def path_to_target(path):
    img = img_to_array(
        load_img(path, target_size=img_size, color_mode="grayscale"))
    img = img.astype("uint8") - 1
    return img

input_imgs = np.zeros((num_imgs,) + img_size + (3,), dtype="float32")
targets = np.zeros((num_imgs,) + img_size + (1,), dtype="uint8")
for i in range(num_imgs):
    input_imgs[i] = path_to_input_image(input_img_paths[i])
    targets[i] = path_to_target(target_paths[i])
```

```

num_val_samples = 1000
train_input_imgs = input_imgs[:-num_val_samples]
train_targets = targets[:-num_val_samples]
val_input_imgs = input_imgs[-num_val_samples:]
val_targets = targets[-num_val_samples:]

```

1.2 Understanding Conv2DTranspose layers

- Padding mode = “same”
- strides = 2 instead of maxpooling - which destroys pixel location!
- Conv2DTranspose: upsampling, increase the spatial size

First half

- Encode images into smaller feature maps
- Each pixel contains a large spatial chunk of the original image
- Kind of compression

Second half

- Learn to upsample
- Kind of inverse transformations

```

[6]: from tensorflow import keras
from tensorflow.keras import layers

def get_model(img_size, num_classes):
    inputs = keras.Input(shape=img_size + (3,))
    x = layers.Rescaling(1./255)(inputs)

    x = layers.Conv2D(64, 3, strides=2, activation="relu", padding="same")(x)
    x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
    x = layers.Conv2D(128, 3, strides=2, activation="relu", padding="same")(x)
    x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
    x = layers.Conv2D(256, 3, strides=2, padding="same", activation="relu")(x)
    x = layers.Conv2D(256, 3, activation="relu", padding="same")(x)

    x = layers.Conv2DTranspose(256, 3, activation="relu", padding="same")(x)
    x = layers.Conv2DTranspose(256, 3, activation="relu", padding="same",
↪strides=2)(x)
    x = layers.Conv2DTranspose(128, 3, activation="relu", padding="same")(x)
    x = layers.Conv2DTranspose(128, 3, activation="relu", padding="same",
↪strides=2)(x)
    x = layers.Conv2DTranspose(64, 3, activation="relu", padding="same")(x)
    x = layers.Conv2DTranspose(64, 3, activation="relu", padding="same",
↪strides=2)(x)

```

```

        outputs = layers.Conv2D(num_classes, 3, activation="softmax",
padding="same")(x)

        model = keras.Model(inputs, outputs)
        return model

model = get_model(img_size=img_size, num_classes=3)
model.summary()

```

Model: "model"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 200, 200, 3)]	0
rescaling (Rescaling)	(None, 200, 200, 3)	0
conv2d (Conv2D)	(None, 100, 100, 64)	1792
conv2d_1 (Conv2D)	(None, 100, 100, 64)	36928
conv2d_2 (Conv2D)	(None, 50, 50, 128)	73856
conv2d_3 (Conv2D)	(None, 50, 50, 128)	147584
conv2d_4 (Conv2D)	(None, 25, 25, 256)	295168
conv2d_5 (Conv2D)	(None, 25, 25, 256)	590080
conv2d_transpose (Conv2DTranspose)	(None, 25, 25, 256)	590080
conv2d_transpose_1 (Conv2DTranspose)	(None, 50, 50, 256)	590080
conv2d_transpose_2 (Conv2DTranspose)	(None, 50, 50, 128)	295040
conv2d_transpose_3 (Conv2DTranspose)	(None, 100, 100, 128)	147584
conv2d_transpose_4 (Conv2DTranspose)	(None, 100, 100, 64)	73792
conv2d_transpose_5 (Conv2DTranspose)	(None, 200, 200, 64)	36928
conv2d_6 (Conv2D)	(None, 200, 200, 3)	1731

```
=====
Total params: 2,880,643
Trainable params: 2,880,643
Non-trainable params: 0
-----
```

```
[7]: import numpy as np
```

```
train_input_imgs = np.array(train_input_imgs, dtype=np.float32)
train_targets = np.array(train_targets, dtype=np.float32)
val_input_imgs = np.array(val_input_imgs, dtype=np.float32)
val_targets = np.array(val_targets, dtype=np.float32)
```

```
[8]: # Compile the model correctly
model.compile(
    optimizer="rmsprop",
    loss="sparse_categorical_crossentropy", # Check if this matches your
    ↪target format
    metrics=["accuracy"] # Added accuracy for better tracking
)

# Define ModelCheckpoint callback
callbacks = [
    keras.callbacks.ModelCheckpoint(
        "oxford_segmentation.h5",
        save_best_only=True
    )
]

# Train the model correctly
history = model.fit(
    train_input_imgs, train_targets, # Ensure these have correct shapes
    epochs=50,
    batch_size=16, # Moved batch_size to correct position
    validation_data=(val_input_imgs, val_targets),
    callbacks=callbacks # Callbacks in the right place
)
```

```
Epoch 1/50
400/400 [=====] - 48s 112ms/step - loss: 0.8909 -
accuracy: 0.6292 - val_loss: 0.8457 - val_accuracy: 0.6929
Epoch 2/50
400/400 [=====] - 42s 106ms/step - loss: 0.7530 -
accuracy: 0.7007 - val_loss: 0.7773 - val_accuracy: 0.6913
Epoch 3/50
400/400 [=====] - 43s 107ms/step - loss: 0.6540 -
accuracy: 0.7353 - val_loss: 0.6435 - val_accuracy: 0.7400
```

Epoch 4/50
400/400 [=====] - 43s 108ms/step - loss: 0.5737 - accuracy: 0.7714 - val_loss: 0.6421 - val_accuracy: 0.7234

Epoch 5/50
400/400 [=====] - 43s 108ms/step - loss: 0.5206 - accuracy: 0.7948 - val_loss: 0.5871 - val_accuracy: 0.7799

Epoch 6/50
400/400 [=====] - 43s 109ms/step - loss: 0.4950 - accuracy: 0.8084 - val_loss: 0.5164 - val_accuracy: 0.8019

Epoch 7/50
400/400 [=====] - 44s 109ms/step - loss: 0.4620 - accuracy: 0.8212 - val_loss: 0.5156 - val_accuracy: 0.7963

Epoch 8/50
400/400 [=====] - 44s 109ms/step - loss: 0.4314 - accuracy: 0.8329 - val_loss: 0.5394 - val_accuracy: 0.7717

Epoch 9/50
400/400 [=====] - 44s 109ms/step - loss: 0.4133 - accuracy: 0.8406 - val_loss: 0.3972 - val_accuracy: 0.8469

Epoch 10/50
400/400 [=====] - 44s 109ms/step - loss: 0.3953 - accuracy: 0.8474 - val_loss: 0.4251 - val_accuracy: 0.8399

Epoch 11/50
400/400 [=====] - 44s 109ms/step - loss: 0.3808 - accuracy: 0.8535 - val_loss: 0.4523 - val_accuracy: 0.8358

Epoch 12/50
400/400 [=====] - 44s 109ms/step - loss: 0.3660 - accuracy: 0.8596 - val_loss: 0.3677 - val_accuracy: 0.8583

Epoch 13/50
400/400 [=====] - 44s 109ms/step - loss: 0.3531 - accuracy: 0.8646 - val_loss: 0.3746 - val_accuracy: 0.8566

Epoch 14/50
400/400 [=====] - 44s 109ms/step - loss: 0.3395 - accuracy: 0.8696 - val_loss: 0.3728 - val_accuracy: 0.8599

Epoch 15/50
400/400 [=====] - 44s 109ms/step - loss: 0.3282 - accuracy: 0.8742 - val_loss: 0.3842 - val_accuracy: 0.8581

Epoch 16/50
400/400 [=====] - 44s 109ms/step - loss: 0.3170 - accuracy: 0.8784 - val_loss: 0.3528 - val_accuracy: 0.8646

Epoch 17/50
400/400 [=====] - 44s 109ms/step - loss: 0.3012 - accuracy: 0.8844 - val_loss: 0.4207 - val_accuracy: 0.8471

Epoch 18/50
400/400 [=====] - 44s 109ms/step - loss: 0.2939 - accuracy: 0.8873 - val_loss: 0.3915 - val_accuracy: 0.8585

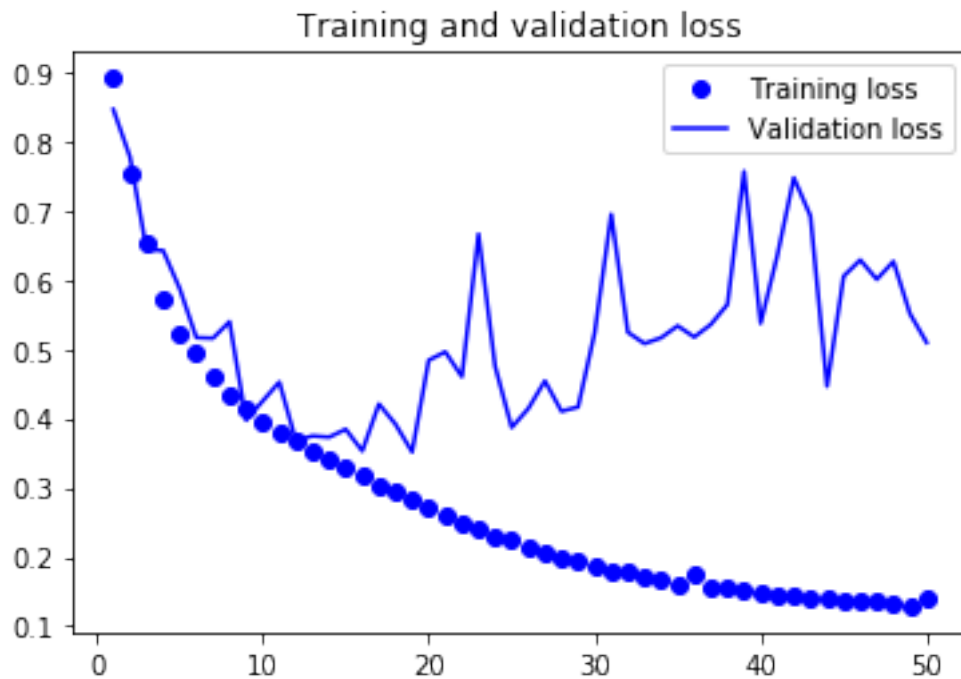
Epoch 19/50
400/400 [=====] - 44s 110ms/step - loss: 0.2823 - accuracy: 0.8917 - val_loss: 0.3514 - val_accuracy: 0.8696

Epoch 20/50
400/400 [=====] - 44s 109ms/step - loss: 0.2717 - accuracy: 0.8958 - val_loss: 0.4838 - val_accuracy: 0.8336
Epoch 21/50
400/400 [=====] - 44s 109ms/step - loss: 0.2593 - accuracy: 0.9004 - val_loss: 0.4959 - val_accuracy: 0.8153
Epoch 22/50
400/400 [=====] - 44s 109ms/step - loss: 0.2499 - accuracy: 0.9039 - val_loss: 0.4601 - val_accuracy: 0.8445
Epoch 23/50
400/400 [=====] - 44s 109ms/step - loss: 0.2415 - accuracy: 0.9074 - val_loss: 0.6658 - val_accuracy: 0.7843
Epoch 24/50
400/400 [=====] - 44s 109ms/step - loss: 0.2308 - accuracy: 0.9113 - val_loss: 0.4753 - val_accuracy: 0.8350
Epoch 25/50
400/400 [=====] - 44s 109ms/step - loss: 0.2237 - accuracy: 0.9137 - val_loss: 0.3868 - val_accuracy: 0.8703
Epoch 26/50
400/400 [=====] - 44s 109ms/step - loss: 0.2147 - accuracy: 0.9171 - val_loss: 0.4140 - val_accuracy: 0.8690
Epoch 27/50
400/400 [=====] - 44s 109ms/step - loss: 0.2041 - accuracy: 0.9207 - val_loss: 0.4540 - val_accuracy: 0.8651
Epoch 28/50
400/400 [=====] - 44s 109ms/step - loss: 0.1995 - accuracy: 0.9231 - val_loss: 0.4102 - val_accuracy: 0.8709
Epoch 29/50
400/400 [=====] - 44s 110ms/step - loss: 0.1956 - accuracy: 0.9246 - val_loss: 0.4163 - val_accuracy: 0.8412
Epoch 30/50
400/400 [=====] - 44s 109ms/step - loss: 0.1870 - accuracy: 0.9273 - val_loss: 0.5234 - val_accuracy: 0.8615
Epoch 31/50
400/400 [=====] - 44s 109ms/step - loss: 0.1799 - accuracy: 0.9299 - val_loss: 0.6945 - val_accuracy: 0.8011
Epoch 32/50
400/400 [=====] - 44s 109ms/step - loss: 0.1792 - accuracy: 0.9304 - val_loss: 0.5242 - val_accuracy: 0.8619
Epoch 33/50
400/400 [=====] - 44s 109ms/step - loss: 0.1709 - accuracy: 0.9335 - val_loss: 0.5078 - val_accuracy: 0.8653
Epoch 34/50
400/400 [=====] - 44s 109ms/step - loss: 0.1677 - accuracy: 0.9344 - val_loss: 0.5162 - val_accuracy: 0.8629
Epoch 35/50
400/400 [=====] - 44s 110ms/step - loss: 0.1616 - accuracy: 0.9368 - val_loss: 0.5335 - val_accuracy: 0.8605

Epoch 36/50
400/400 [=====] - 44s 110ms/step - loss: 0.1735 -
accuracy: 0.9355 - val_loss: 0.5172 - val_accuracy: 0.8684
Epoch 37/50
400/400 [=====] - 45s 112ms/step - loss: 0.1569 -
accuracy: 0.9385 - val_loss: 0.5355 - val_accuracy: 0.8574
Epoch 38/50
400/400 [=====] - 45s 113ms/step - loss: 0.1560 -
accuracy: 0.9389 - val_loss: 0.5643 - val_accuracy: 0.8604
Epoch 39/50
400/400 [=====] - 45s 112ms/step - loss: 0.1534 -
accuracy: 0.9401 - val_loss: 0.7564 - val_accuracy: 0.8579
Epoch 40/50
400/400 [=====] - 45s 113ms/step - loss: 0.1485 -
accuracy: 0.9418 - val_loss: 0.5371 - val_accuracy: 0.8663
Epoch 41/50
400/400 [=====] - 45s 113ms/step - loss: 0.1443 -
accuracy: 0.9434 - val_loss: 0.6345 - val_accuracy: 0.8627
Epoch 42/50
400/400 [=====] - 45s 114ms/step - loss: 0.1440 -
accuracy: 0.9436 - val_loss: 0.7471 - val_accuracy: 0.8596
Epoch 43/50
400/400 [=====] - 45s 114ms/step - loss: 0.1416 -
accuracy: 0.9444 - val_loss: 0.6920 - val_accuracy: 0.8553
Epoch 44/50
400/400 [=====] - 45s 114ms/step - loss: 0.1391 -
accuracy: 0.9454 - val_loss: 0.4462 - val_accuracy: 0.8645
Epoch 45/50
400/400 [=====] - 45s 114ms/step - loss: 0.1357 -
accuracy: 0.9465 - val_loss: 0.6050 - val_accuracy: 0.8664
Epoch 46/50
400/400 [=====] - 45s 114ms/step - loss: 0.1362 -
accuracy: 0.9468 - val_loss: 0.6285 - val_accuracy: 0.8686
Epoch 47/50
400/400 [=====] - 45s 114ms/step - loss: 0.1377 -
accuracy: 0.9466 - val_loss: 0.6005 - val_accuracy: 0.8425
Epoch 48/50
400/400 [=====] - 45s 114ms/step - loss: 0.1346 -
accuracy: 0.9479 - val_loss: 0.6264 - val_accuracy: 0.8469
Epoch 49/50
400/400 [=====] - 45s 114ms/step - loss: 0.1292 -
accuracy: 0.9490 - val_loss: 0.5507 - val_accuracy: 0.8159
Epoch 50/50
400/400 [=====] - 45s 114ms/step - loss: 0.1409 -
accuracy: 0.9484 - val_loss: 0.5094 - val_accuracy: 0.8643

```
[9]: epochs = range(1, len(history.history["loss"]) + 1)
loss = history.history["loss"]
val_loss = history.history["val_loss"]
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.legend()
```

[9]: <matplotlib.legend.Legend at 0x7faaf803a550>



```
[10]: from tensorflow.keras.utils import array_to_img

model = keras.models.load_model("oxford_segmentation.h5")

i = 4
test_image = val_input_imgs[i]
plt.axis("off")
plt.imshow(array_to_img(test_image))

mask = model.predict(np.expand_dims(test_image, 0))[0]

def display_mask(pred):
    mask = np.argmax(pred, axis=-1)
```

```
mask *= 127
plt.axis("off")
plt.imshow(mask)

display_mask(mask)
```

1/1 [=====] - 1s 579ms/step

